

Binary Heap

Dr. Md. Shamsuzzoha Bayzid



Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

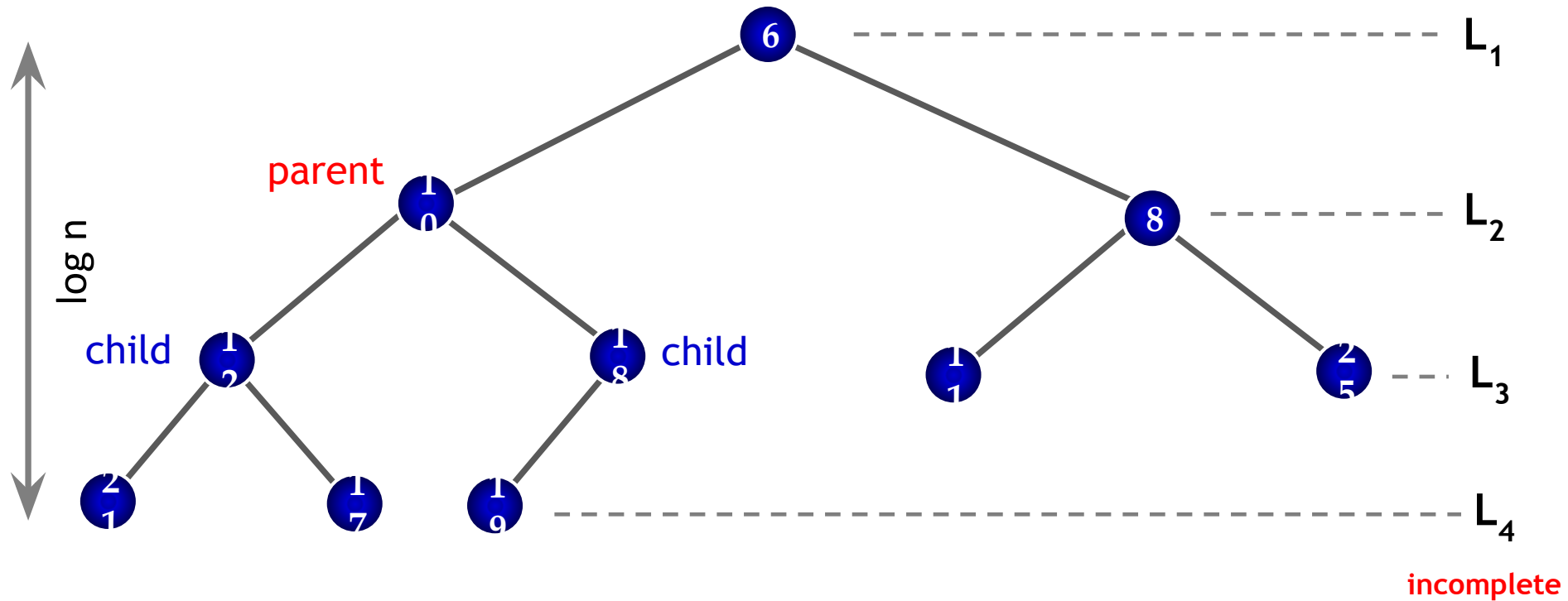
Binary Heap

- **Structural property:** Almost complete binary tree
 - Complete on all levels, except the last
- **Order (heap) property:** Min/Max Heap
 - Parent is less/greater than the children

Binary Heap

- Applications: Priority Queue
 - Dijkstra's shortest path algorithm
 - Prim's MST algorithm
 - Heapsort

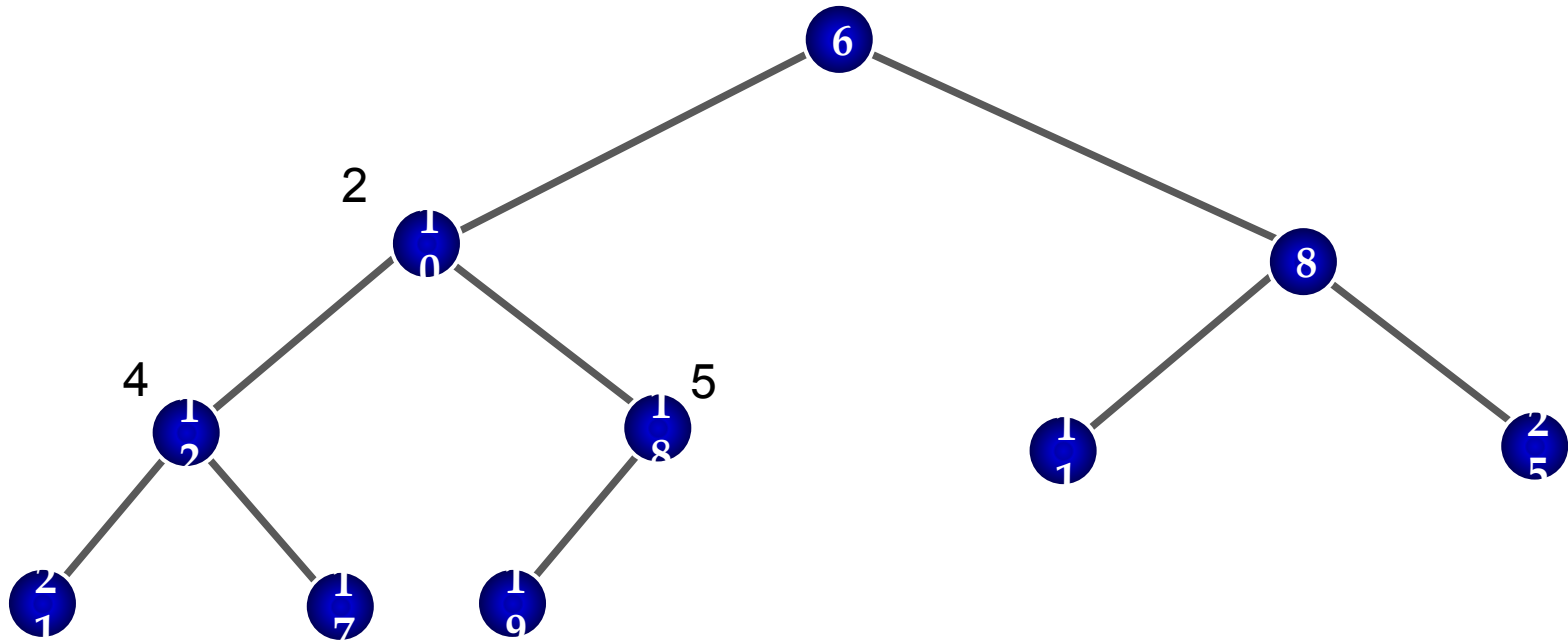
Min Heap



Binary Heap: Properties

- min/max element in the root
- Heap with n elements has $\text{floor}(\log n)$ height
- Maintain pointer to root node
- Maintain pointers to children nodes

Min Heap: Array Implementation

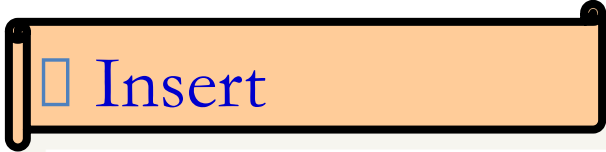


i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	6	10	8	12	18	11	25	21	17	19						

Binary Heap: Properties

- min/max element in the root
- Heap with n elements has $\text{floor}(\log n)$ height
- Maintain pointer to root node
- Maintain pointers to children nodes
- $\text{parent}(i) = \text{floor}(i/2)$
 $\text{left}(i) = 2i$
 $\text{right}(i) = 2i+1$

Binary Heap: basic properties

An orange scroll with a black border and a small black handle on the left side.

- Insert

- Extract min

- Decrease key

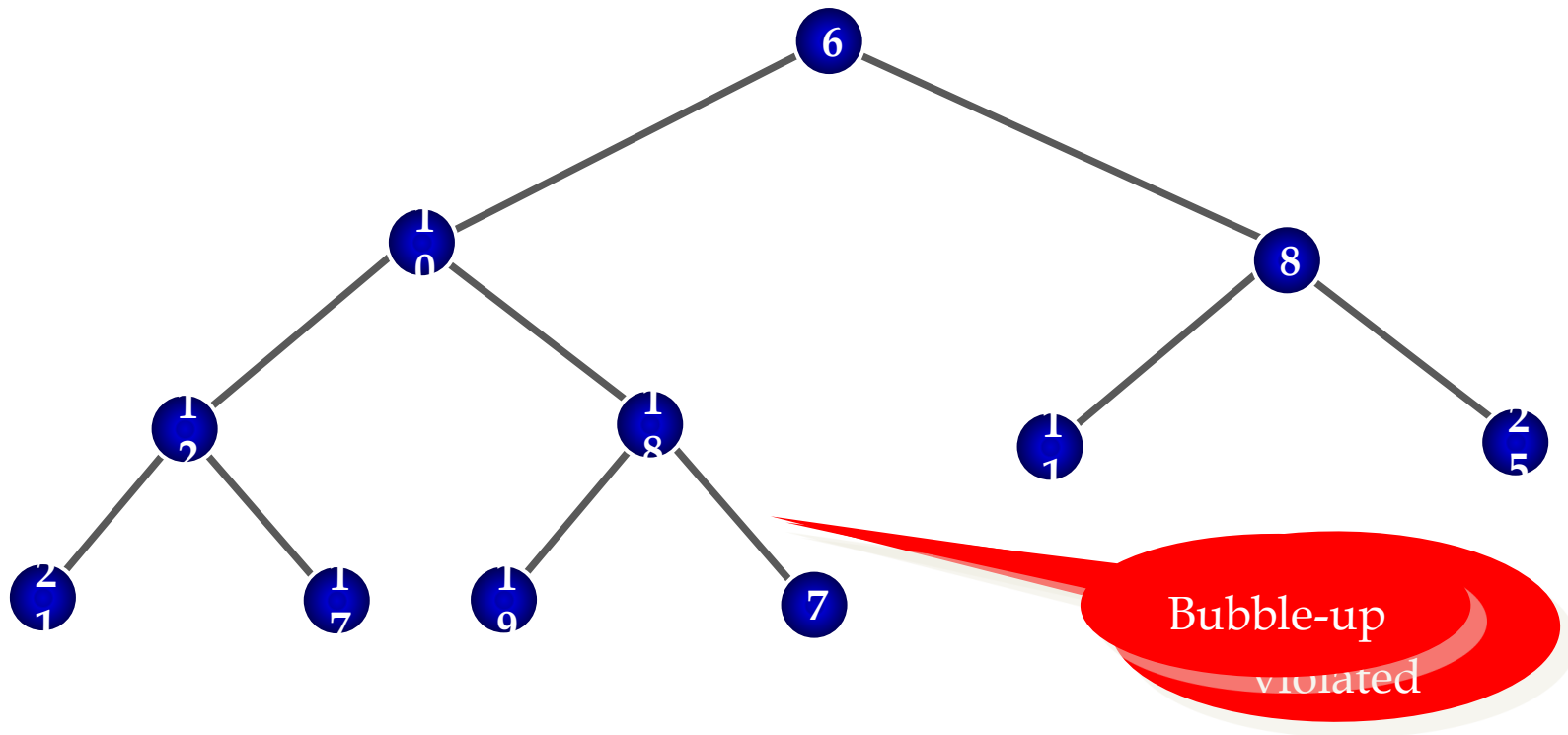
- Find min

- Delete

Min Heap: Insertion

- Insert 7

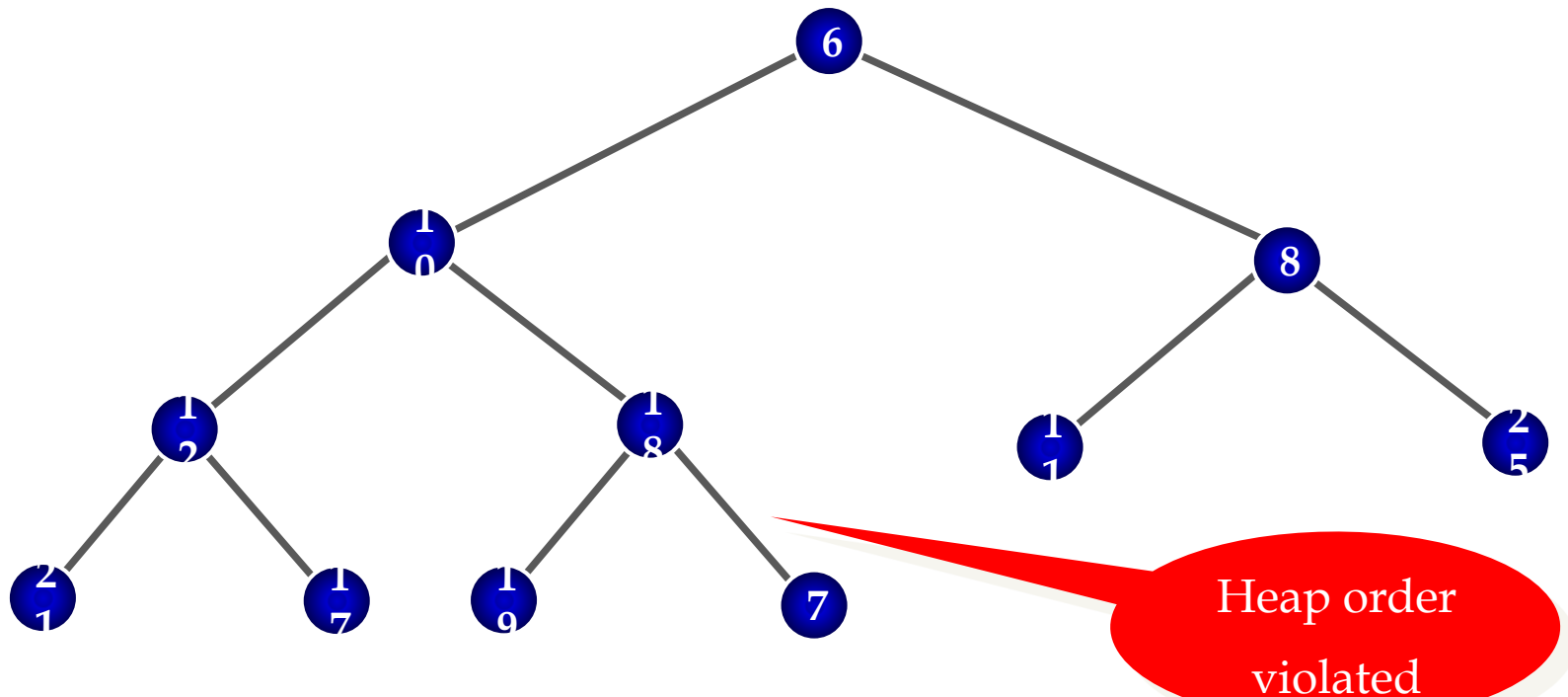
- Add the new element at the **end**
- Bubble up **until** the heap is **ordered**



Min Heap: Insertion

- Insert 7

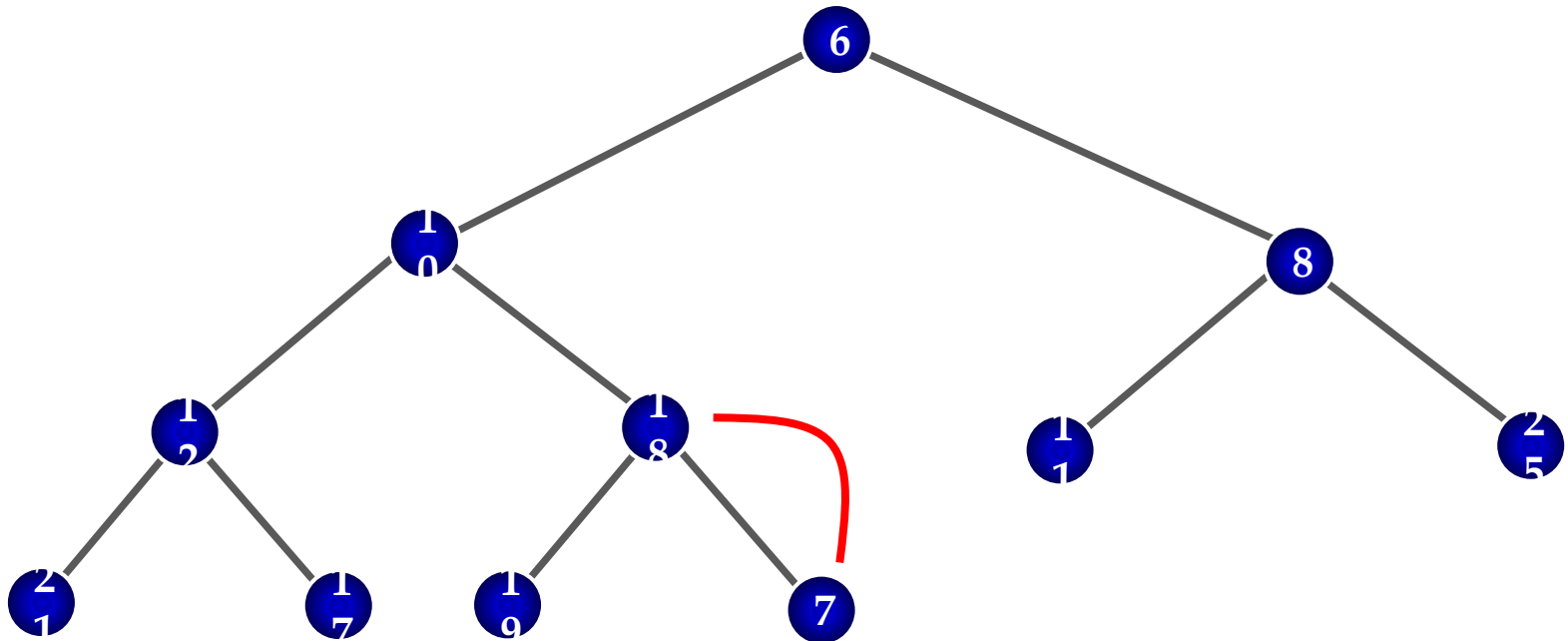
- Add the new element at the **end**
- Bubble up **until** the heap is **ordered**



Min Heap: Insertion

- Insert 7

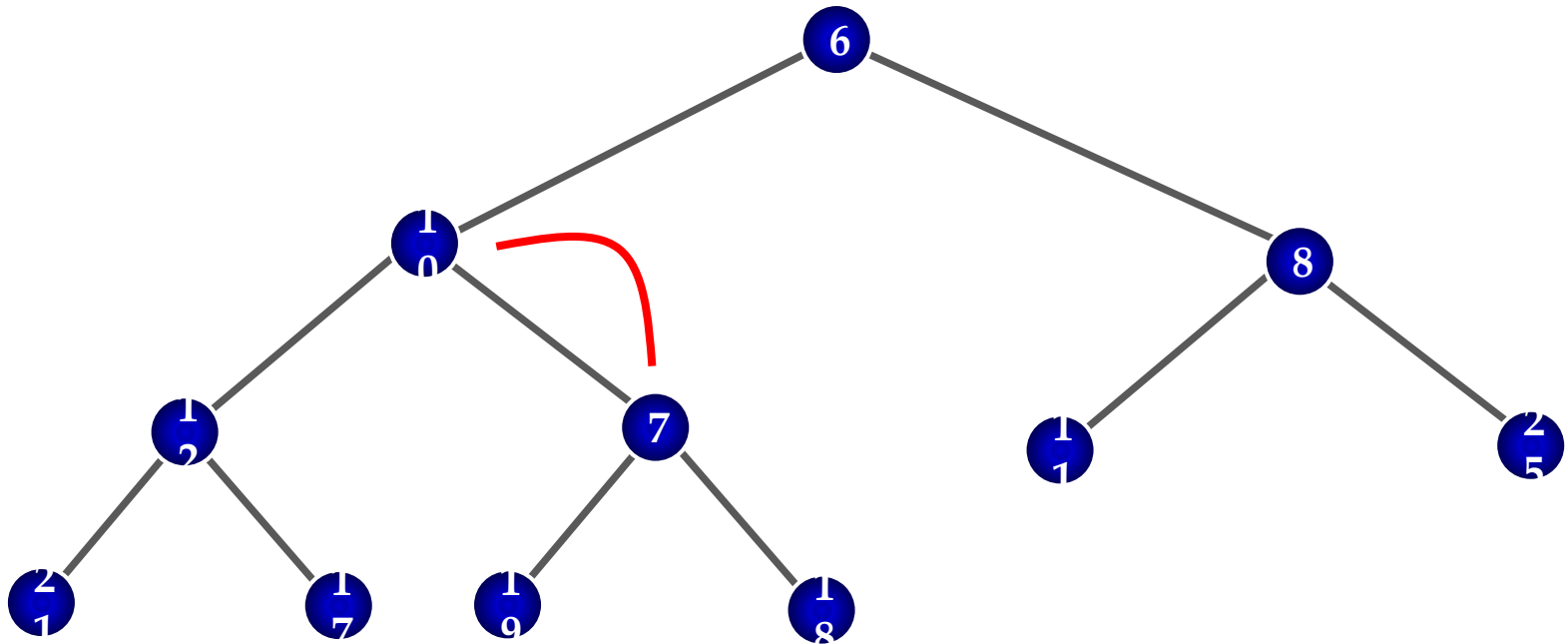
- Add the new element at the **end**
- Bubble up **until** the heap is **ordered**



Min Heap: Insertion

- Insert 7

- Add the new element at the **end**
- Bubble up **until** the heap is **ordered**



Binary Heap: basic properties

□ Insert

□ Extract min

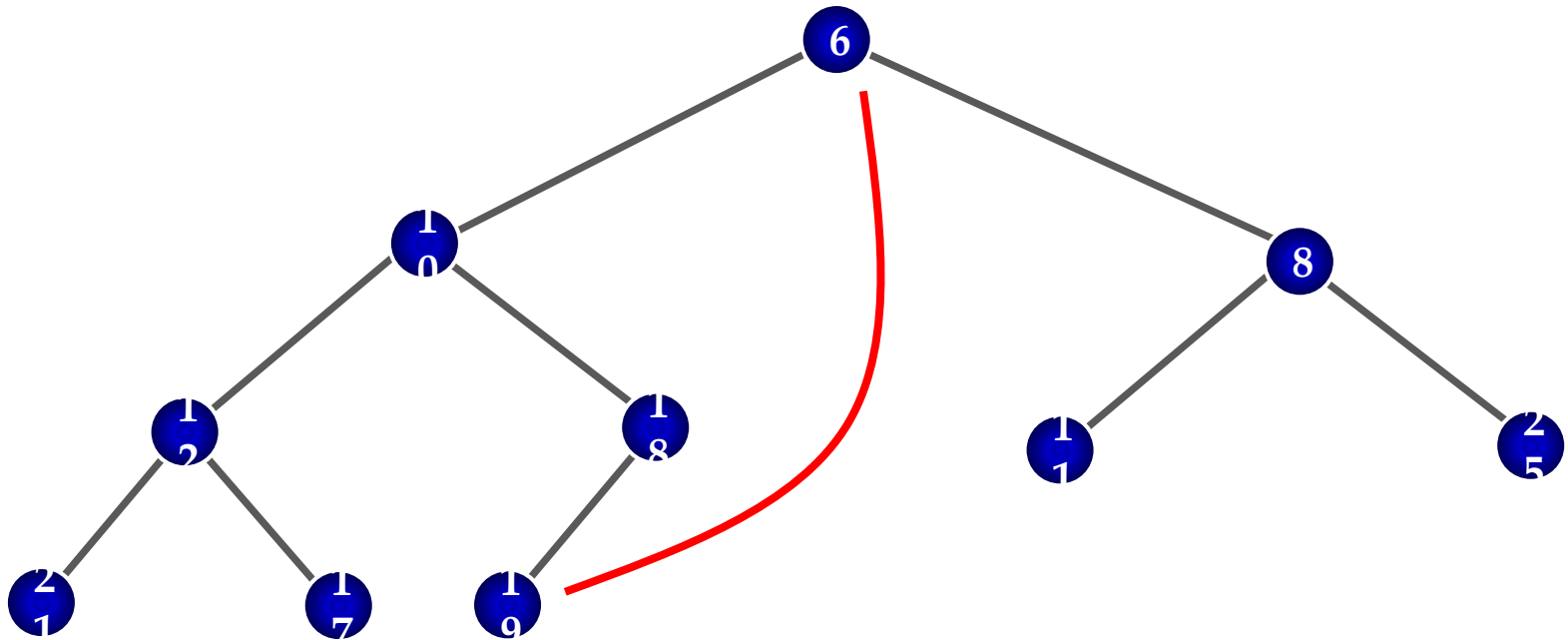
□ Decrease key

□ Find min

□ Delete

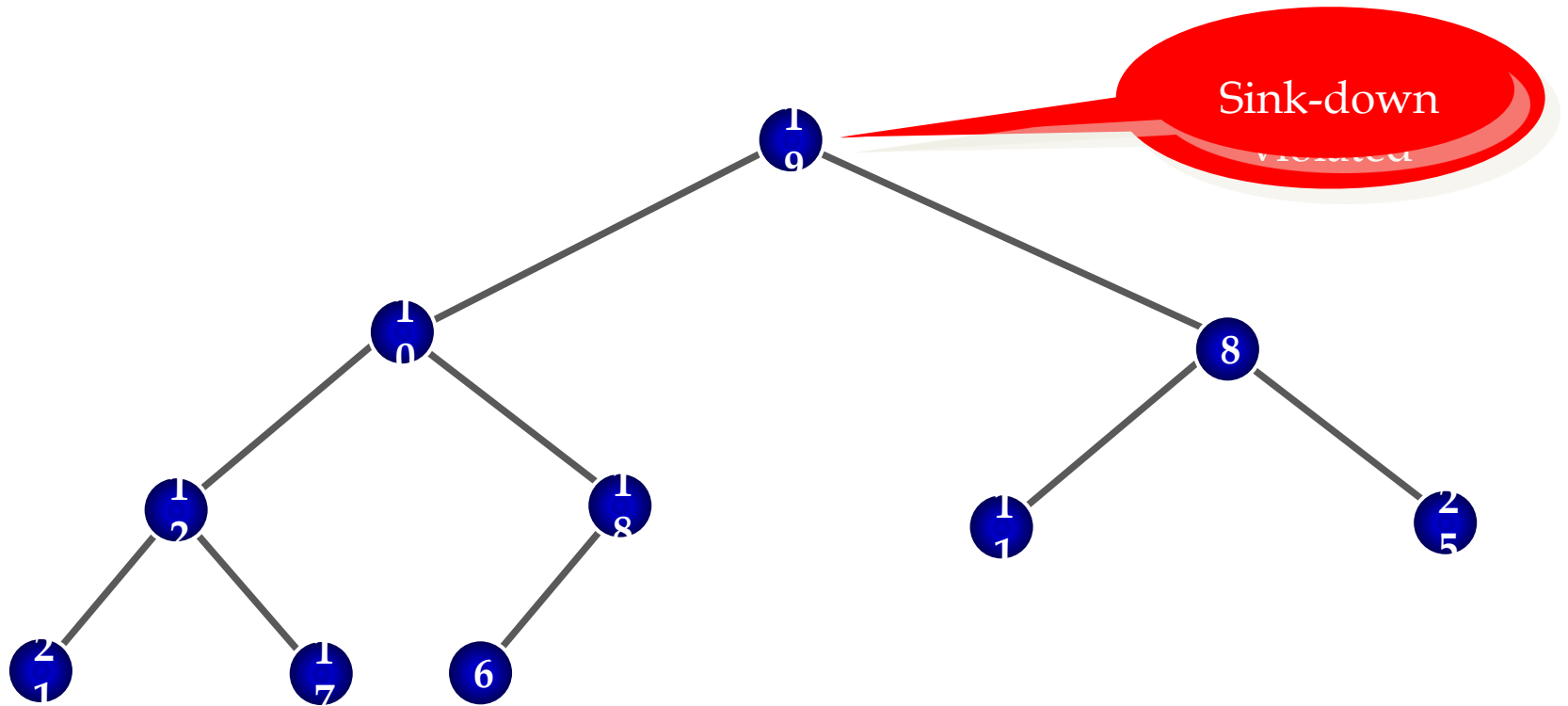
Min Heap: Extract min

- Exchange root with the last node (right-most leaf)
- Sink down until the heap property is retained



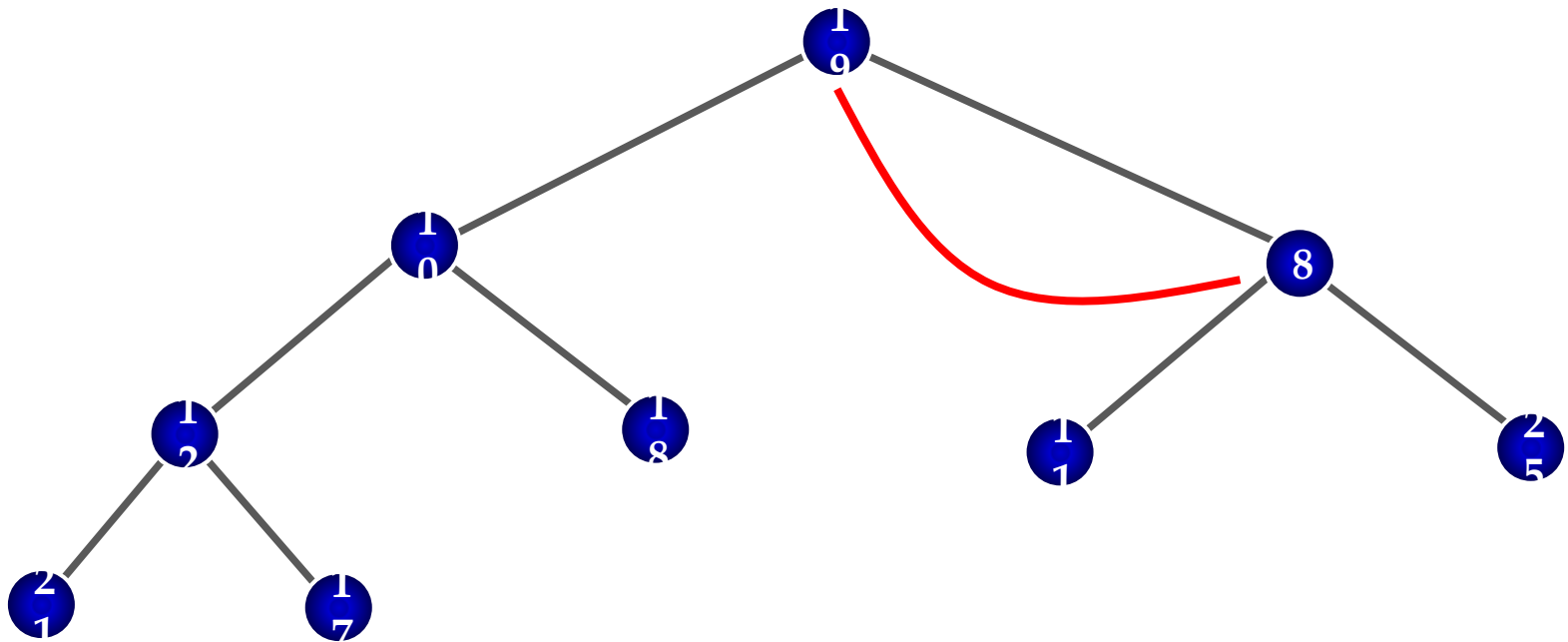
Min Heap: Extract min

- Exchange root with the last node (right-most leaf)
- Sink down until the heap property is retained



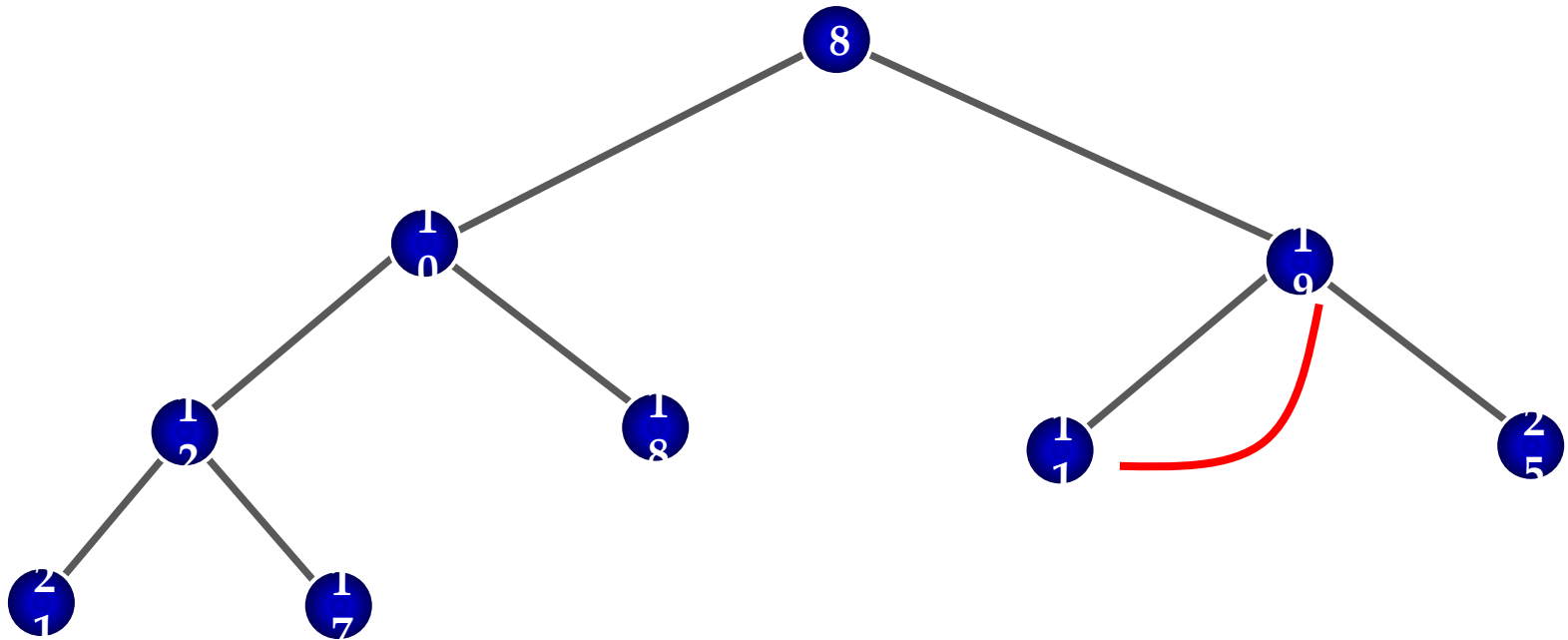
Min Heap: Extract min

- Exchange root with the last node (right-most leaf)
- Sink down until the heap property is retained



Min Heap: Extract min

- Exchange root with the last node (right-most leaf)
- Sink down until the heap property is retained

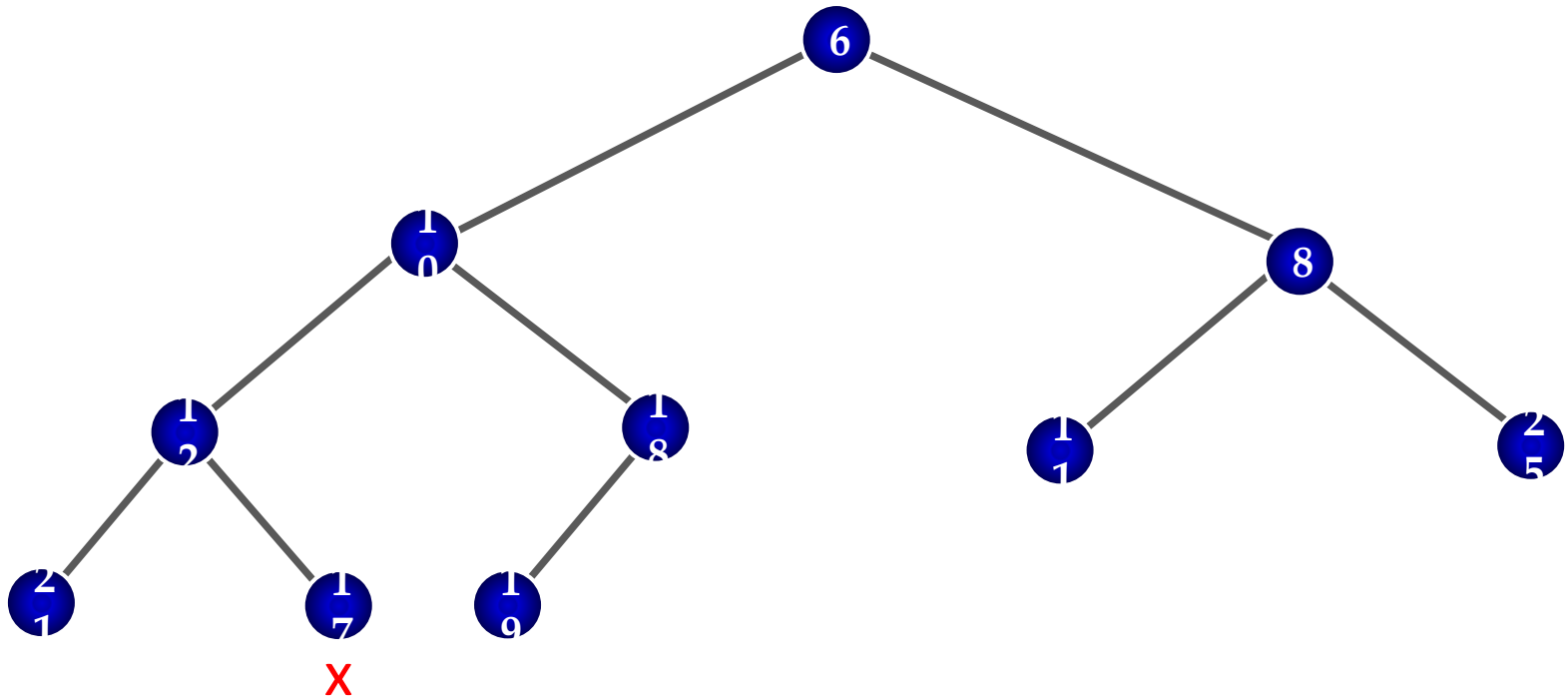


Binary Heap: basic properties

- Insert
- Extract min
- Decrease key
- Find min
- Delete

Min Heap: Decrease key

- Insert x ($x = 17$ in this example)
 - Decrease x
 - Bubble up **until** the heap is ordered



Binary Heap: basic properties

- Insert
- Extract min
- Decrease key

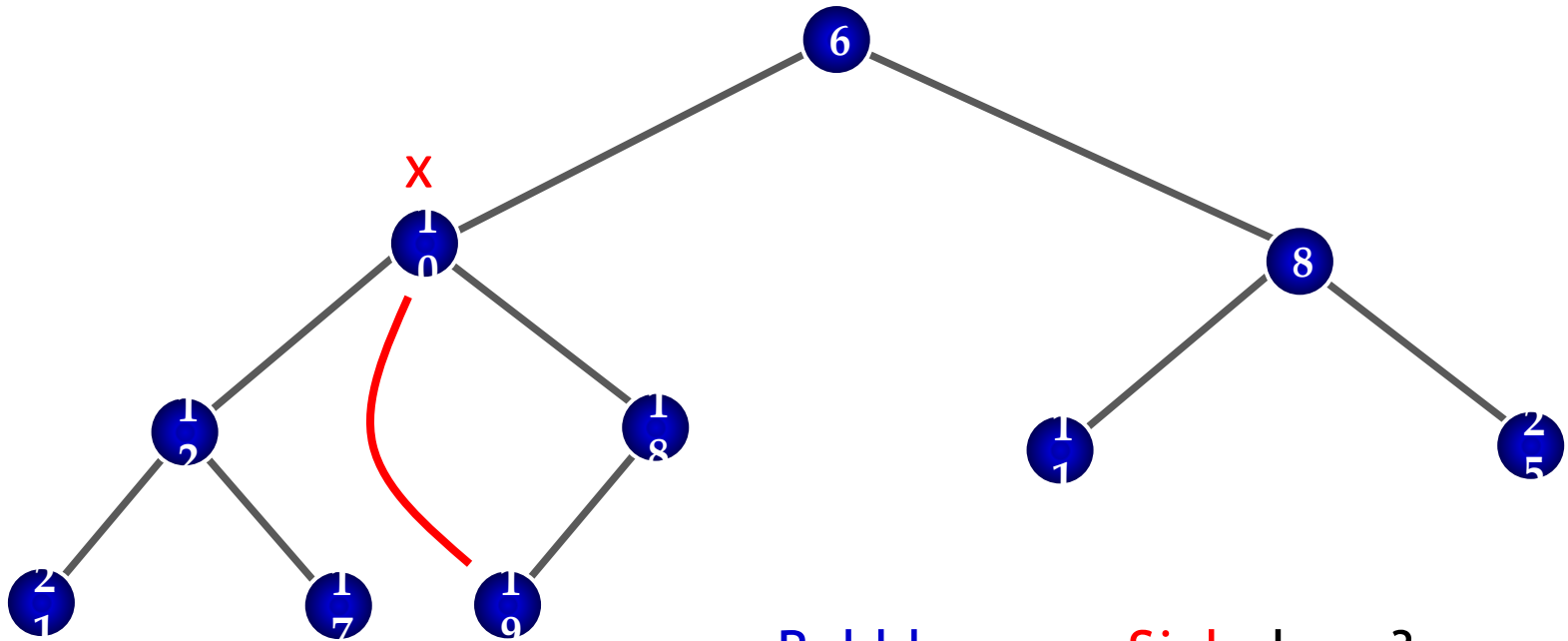
□ Find min

□ Delete

Just return the **root**!

Min Heap: Delete

- Delete x ($x = 10$ in this example)
 - Exchange with the last node
 - Bubble up/sink down until the heap is ordered



Bubble up or Sink down?

What if $x = 11$?

Min Heap: Running time

Operation	Running time
Insert	$O(\log n)$
Decrease key	$O(\log n)$
Extract min	$O(\log n)$
Delete	$O(\log n)$
Find min	$O(1)$

Building a heap

- Given n elements, **construct** a binary a heap

- **Method 1:** **Perform** n **insert** operations

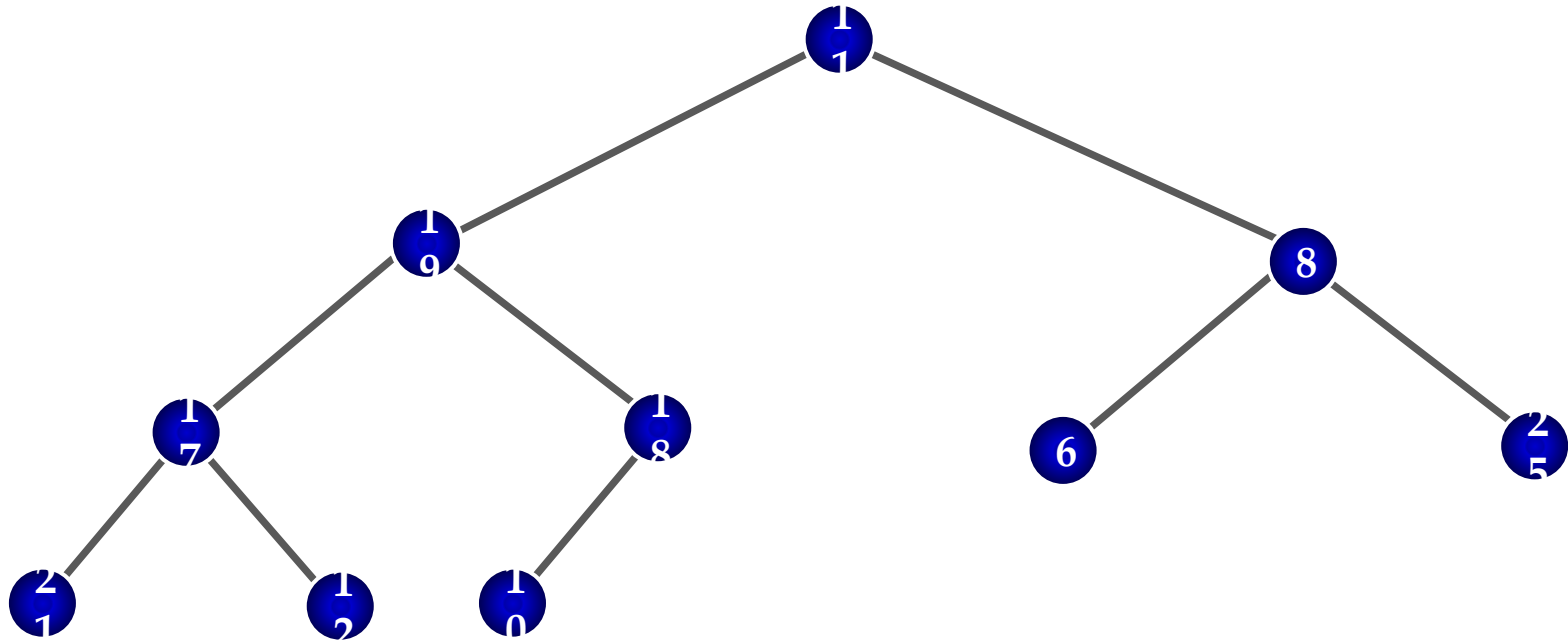
- Running time: $O(n \log n)$

- **Method 2:** **Construct** a binary tree; for $i = n$ to 1 , repeatedly exchange the element in node i with its smaller child until the heap property is achieved (i.e., **perform heapify** for each node in a **bottom-up** fashion)

Heapify

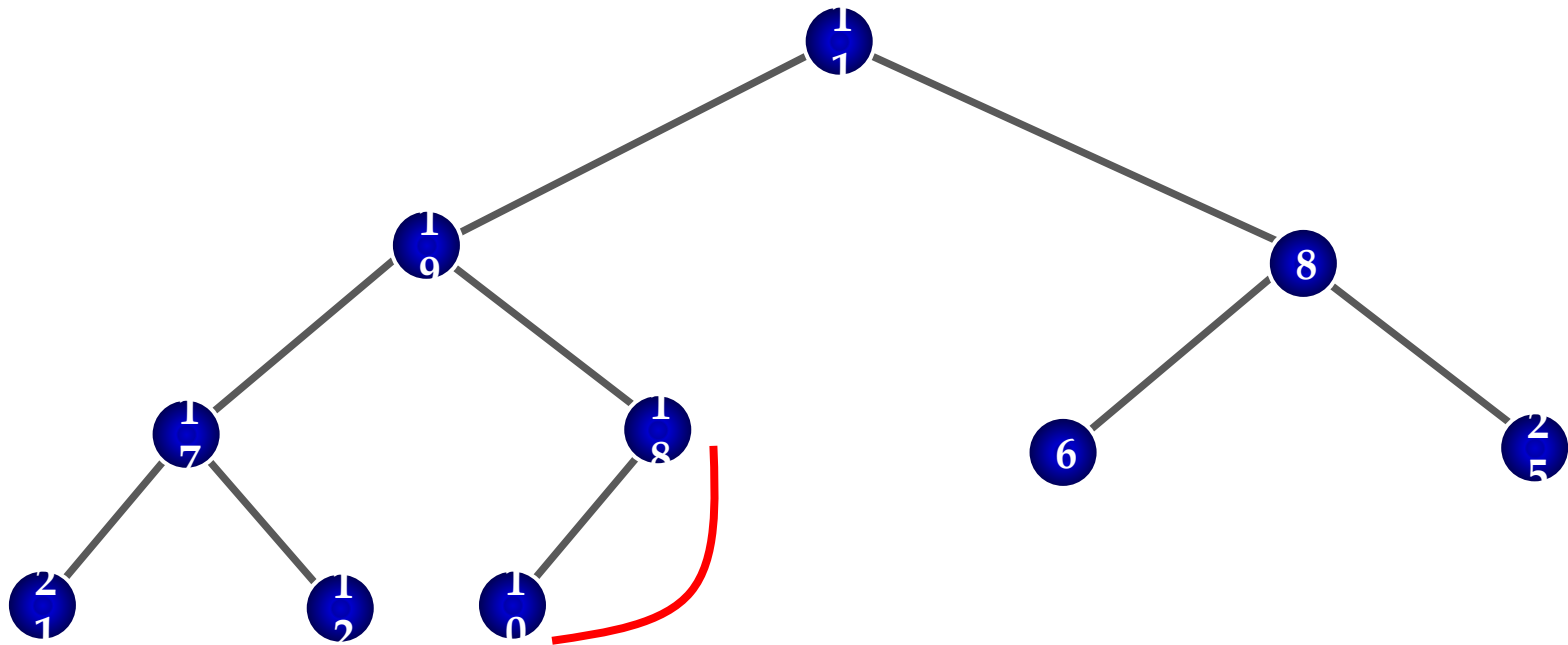
Repeatedly **exchanging** an element with its **smaller** child
until the **heap property** is **achieved**

Constructing a Heap using heapify



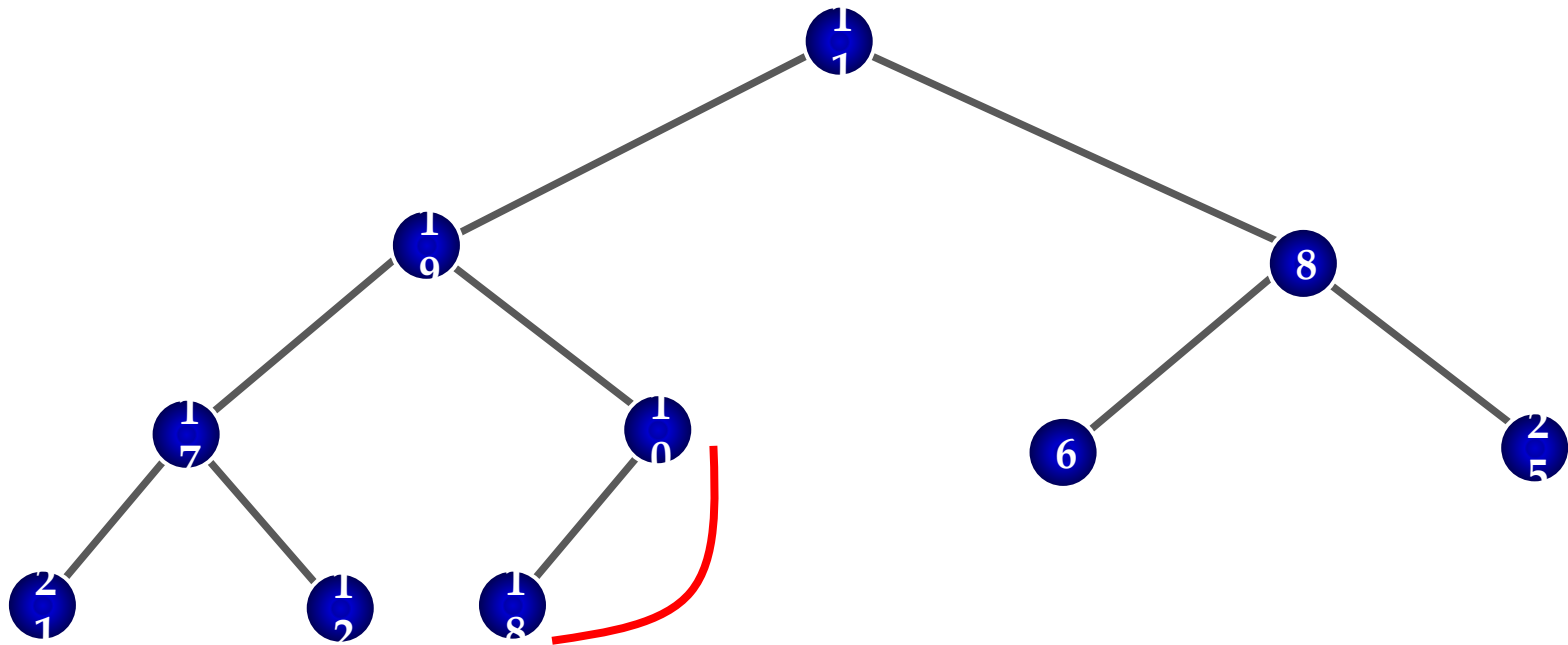
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	11	19	8	17	18	6	25	21	12	10						

Constructing a Heap using heapify



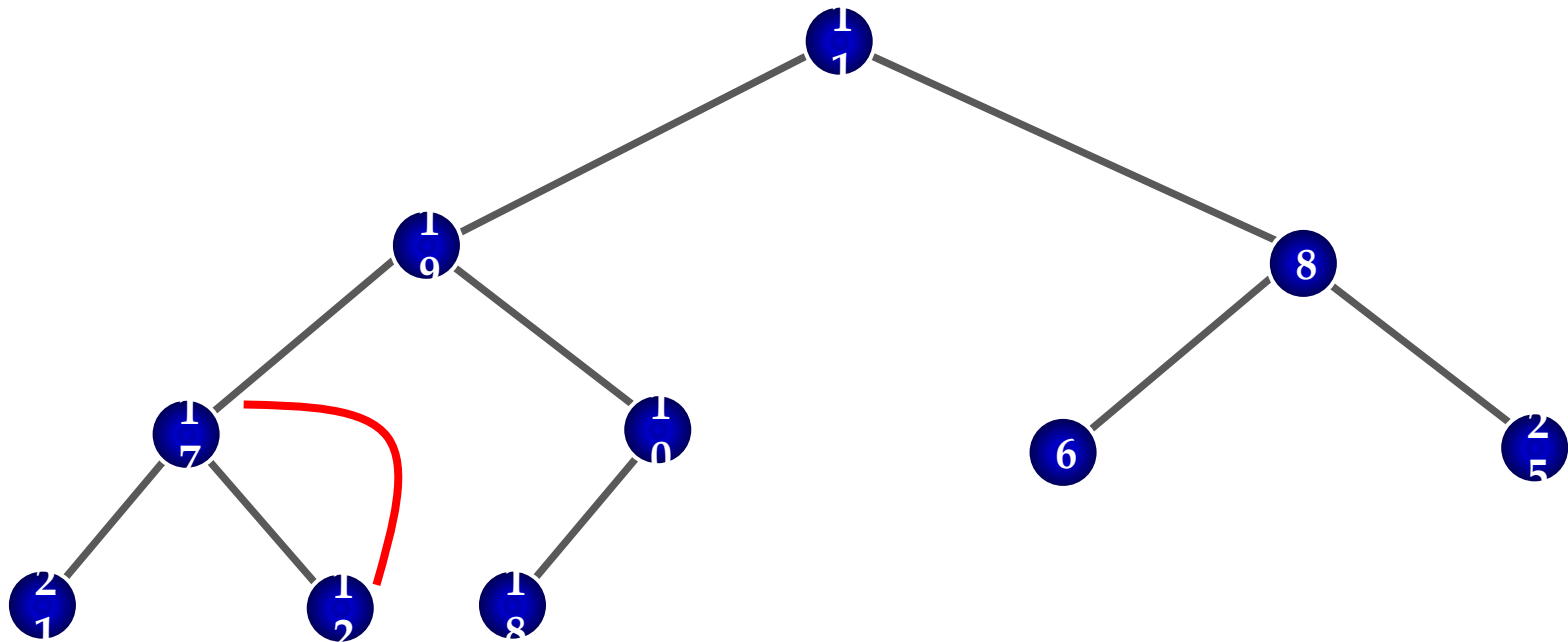
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	11	19	8	17	18	6	25	21	12	10						

Constructing a Heap using heapify



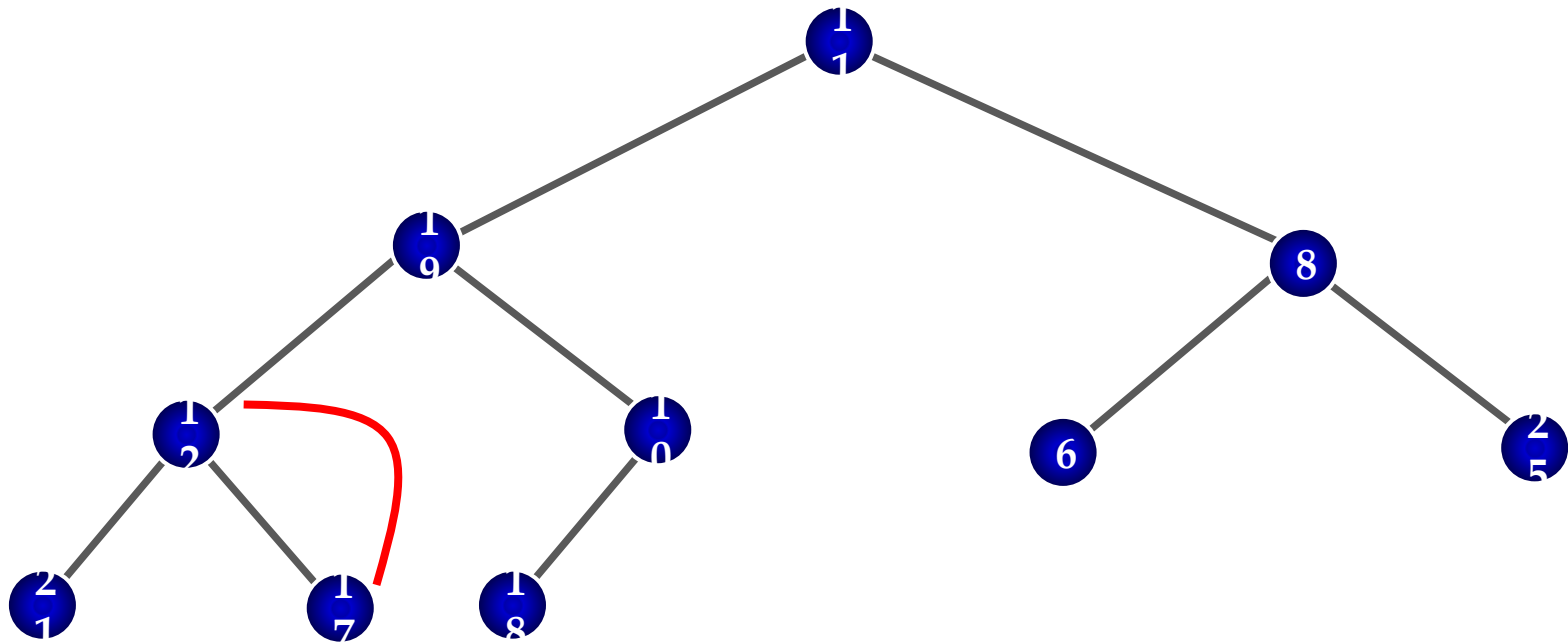
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	11	19	8	17	10	6	25	21	12	18						

Constructing a Heap using heapify



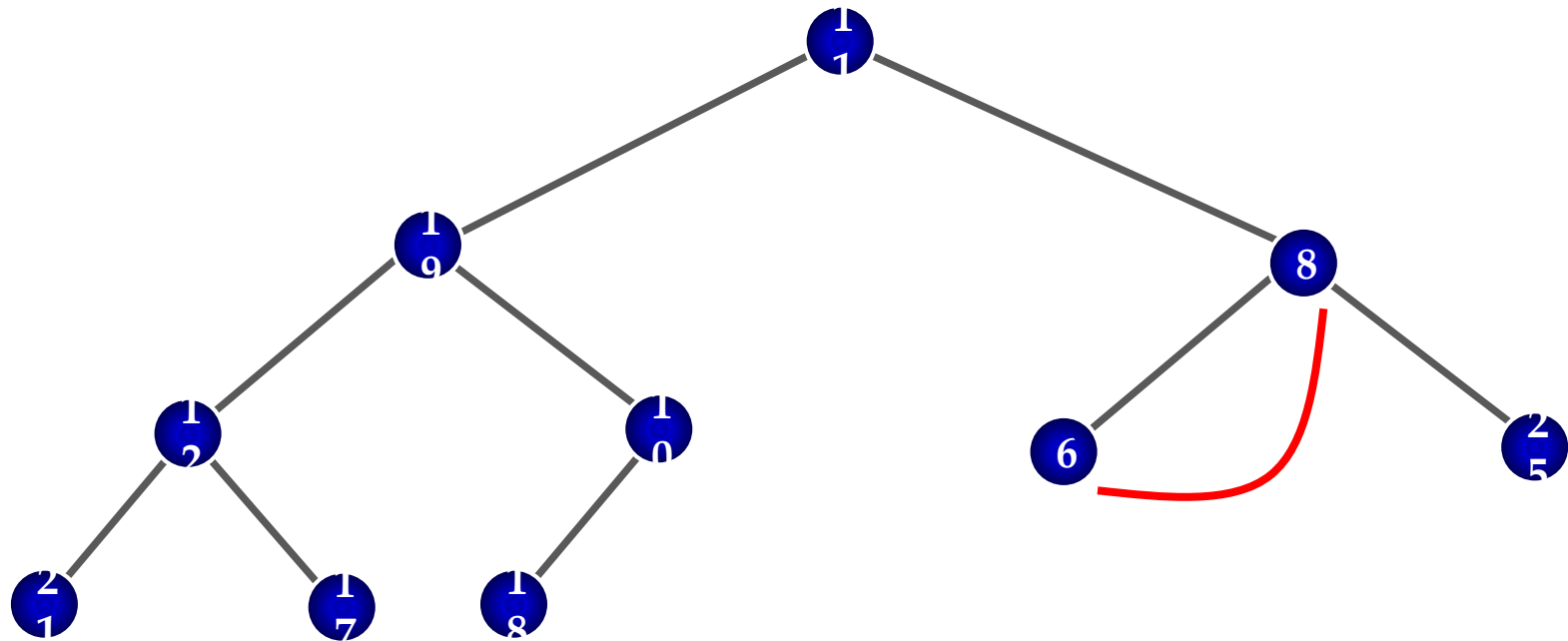
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	11	19	8	17	10	6	25	21	12	18						

Constructing a Heap using heapify



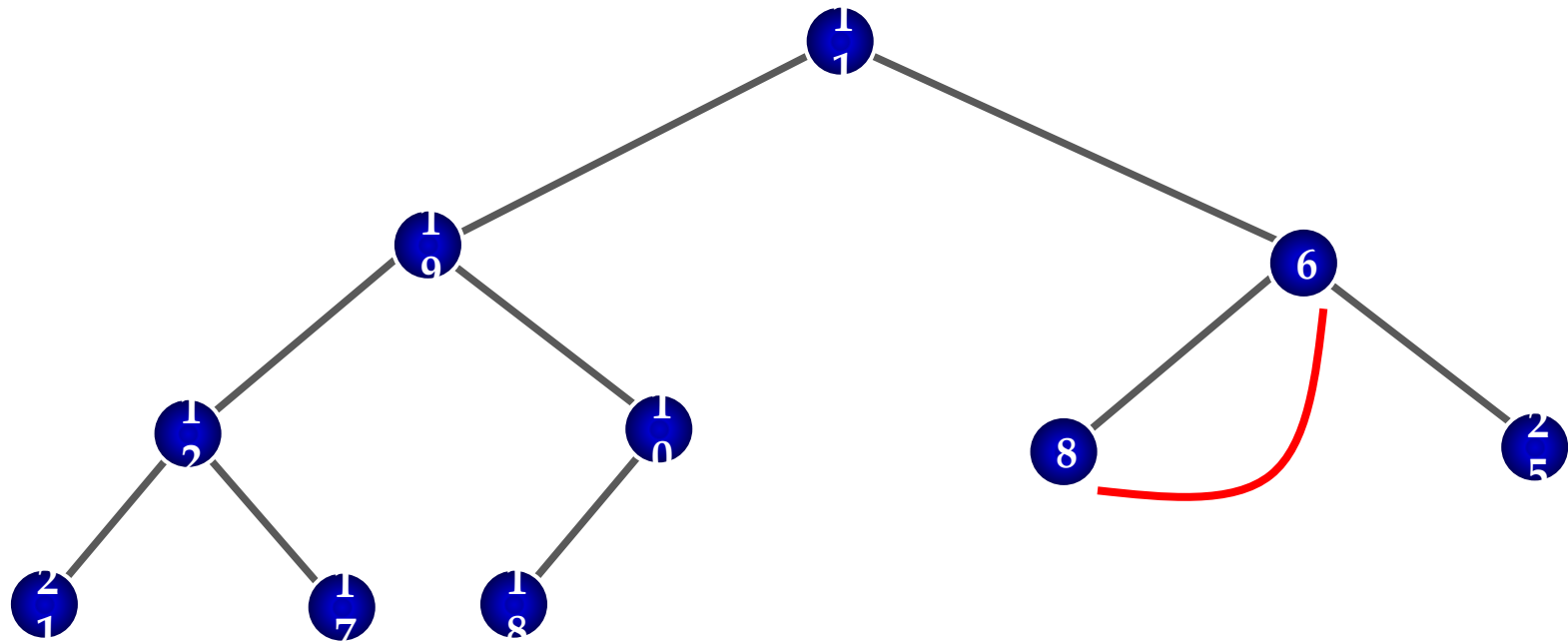
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	11	19	8	12	10	6	25	21	17	18						

Constructing a Heap using heapify



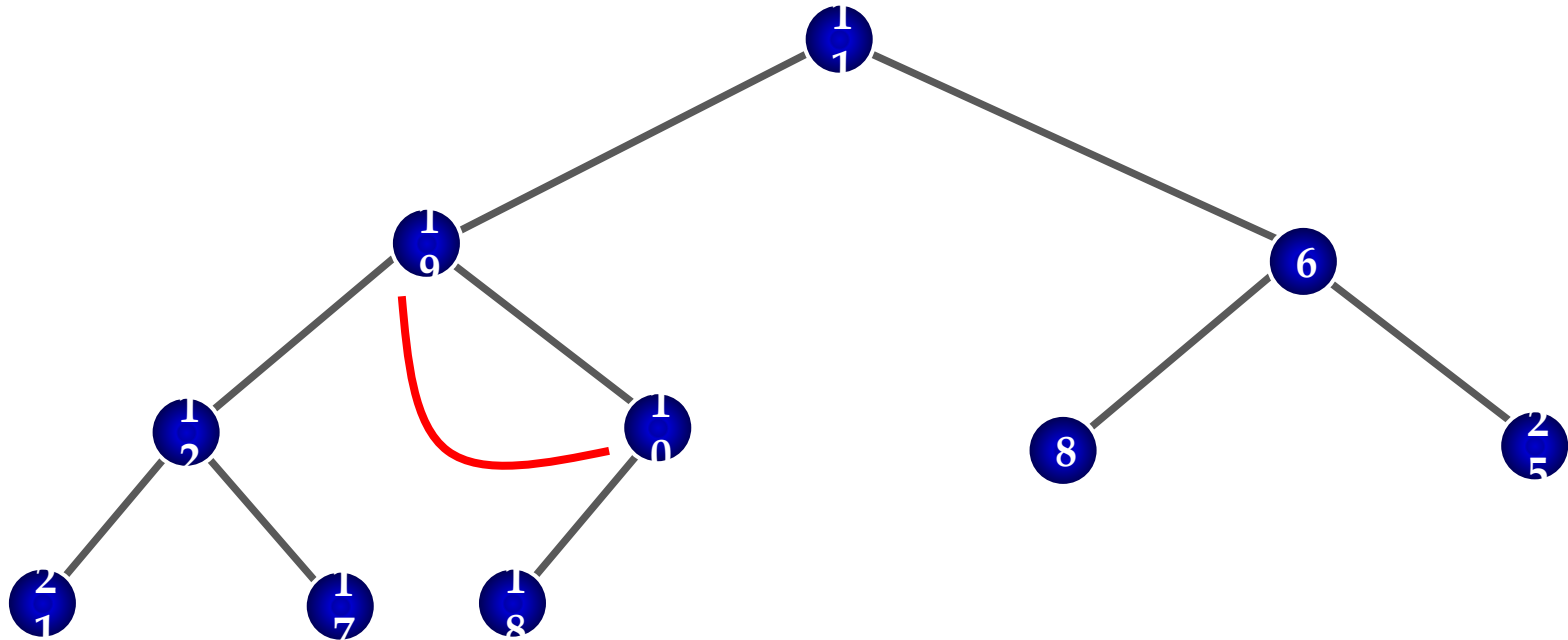
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	11	19	8	12	10	6	25	21	17	18						

Constructing a Heap using heapify



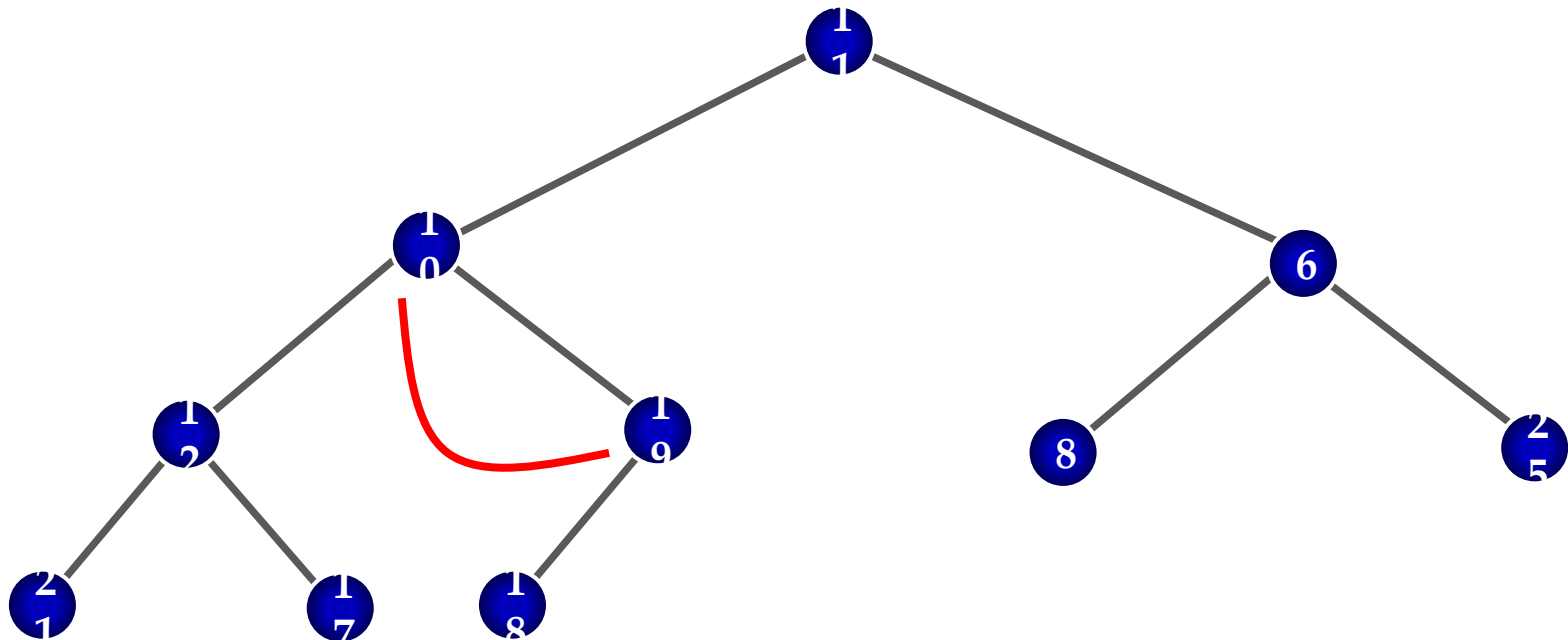
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	11	19	6	12	10	8	25	21	17	18						

Constructing a Heap using heapify



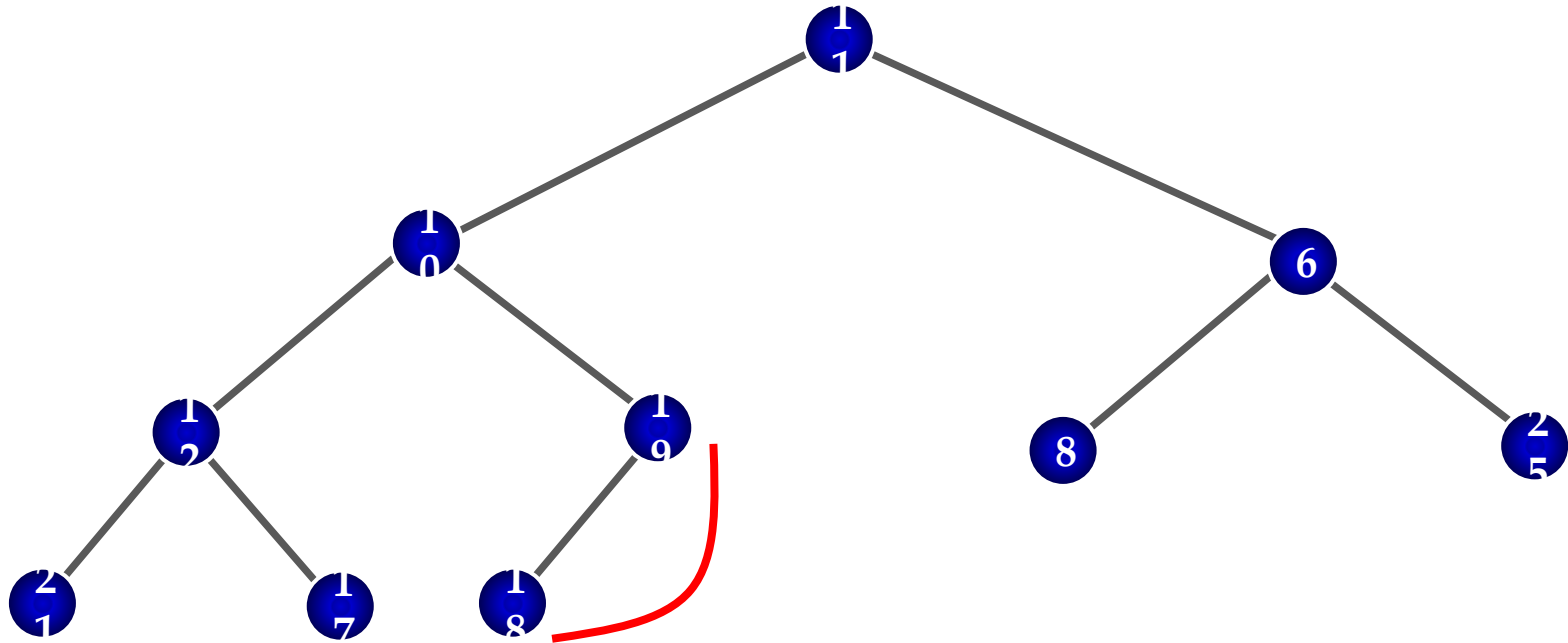
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	11	19	6	12	10	8	25	21	17	18						

Constructing a Heap using heapify



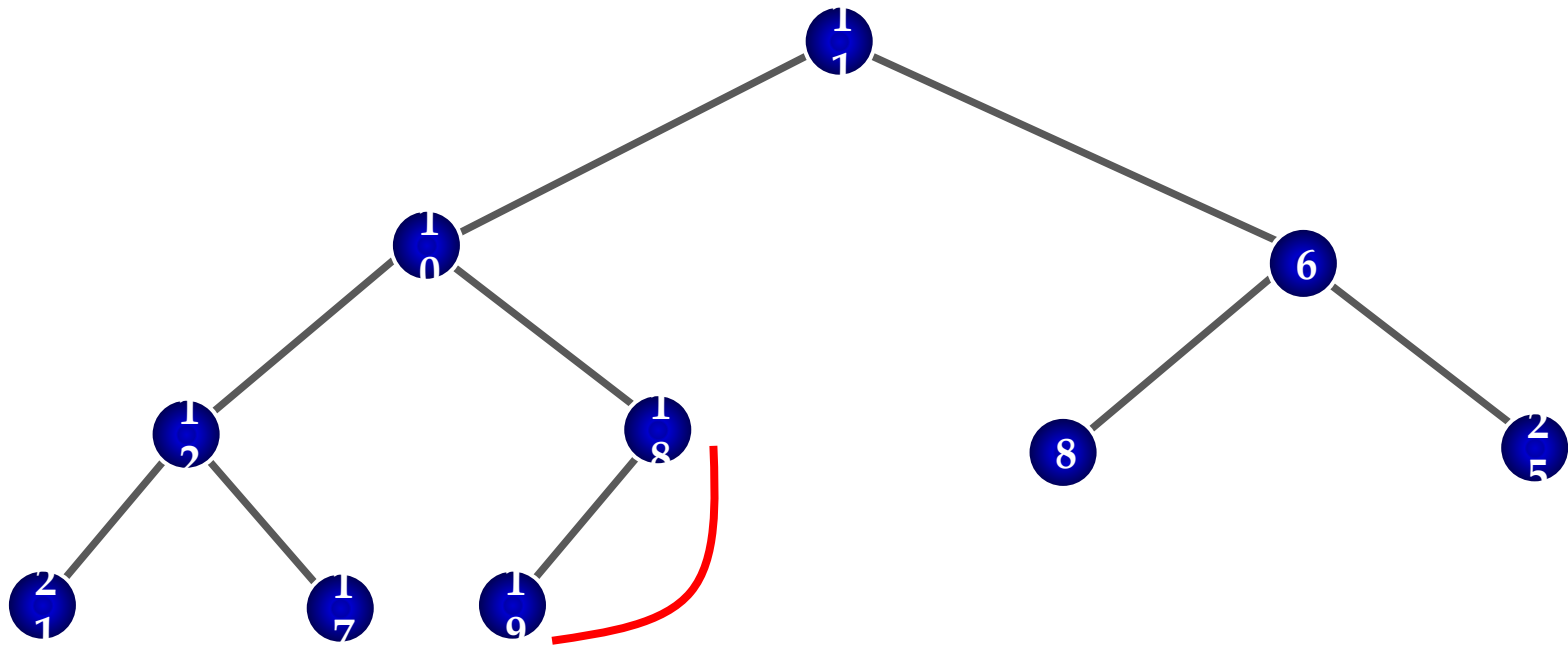
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	11	10	6	12	19	8	25	21	17	18						

Constructing a Heap using heapify



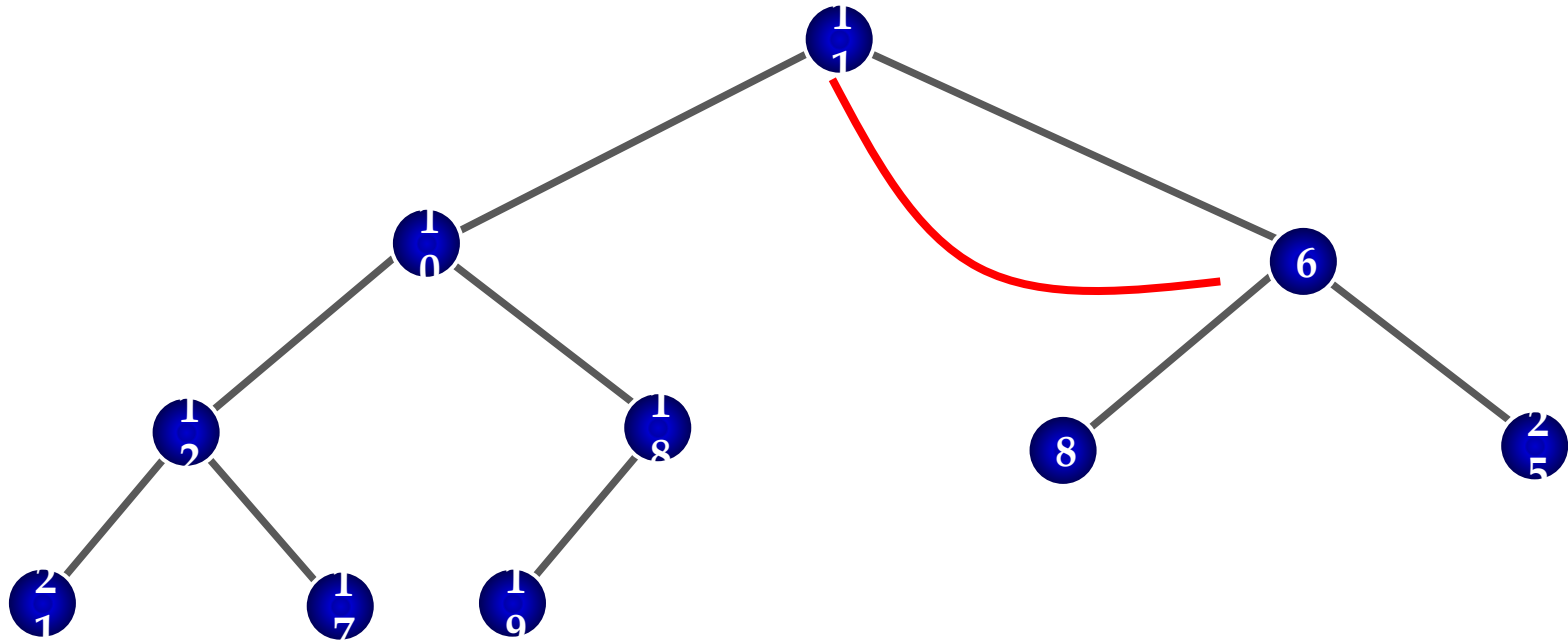
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	11	10	6	12	19	8	25	21	17	18						

Constructing a Heap using heapify



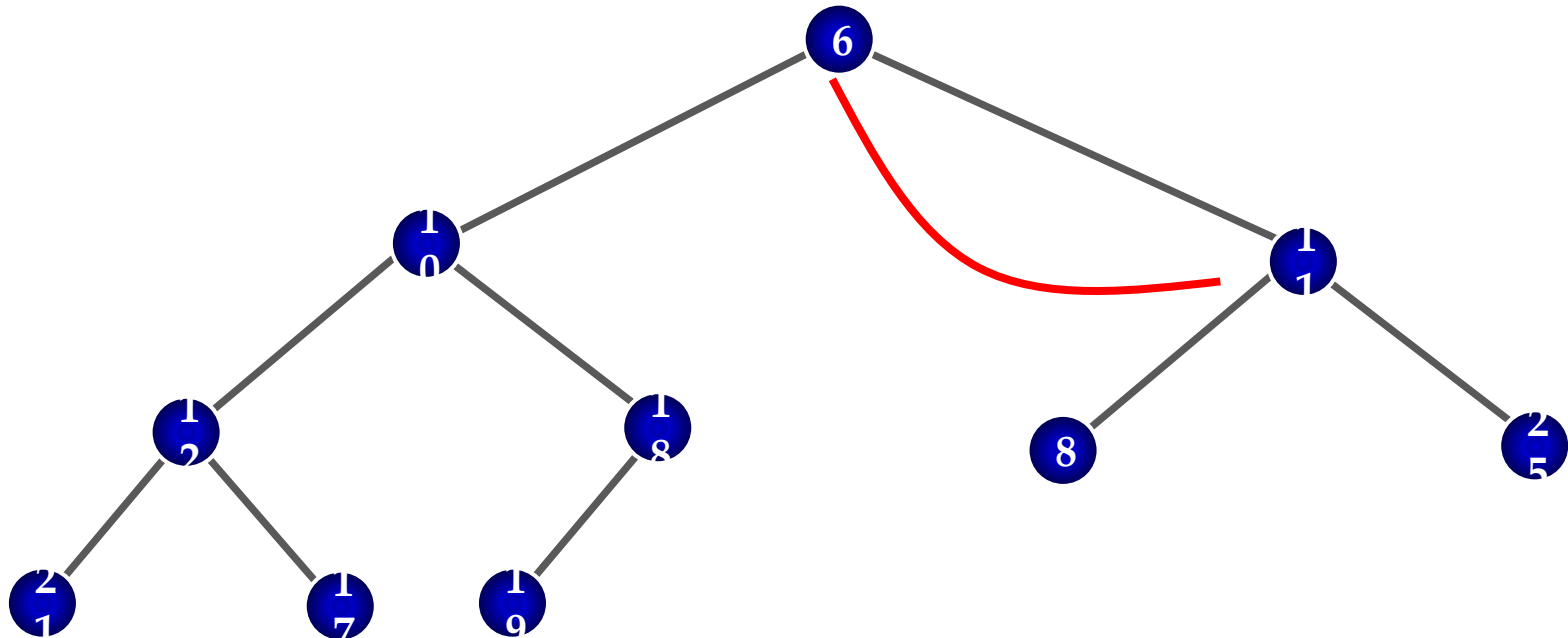
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	11	10	6	12	18	8	25	21	17	19						

Constructing a Heap using heapify



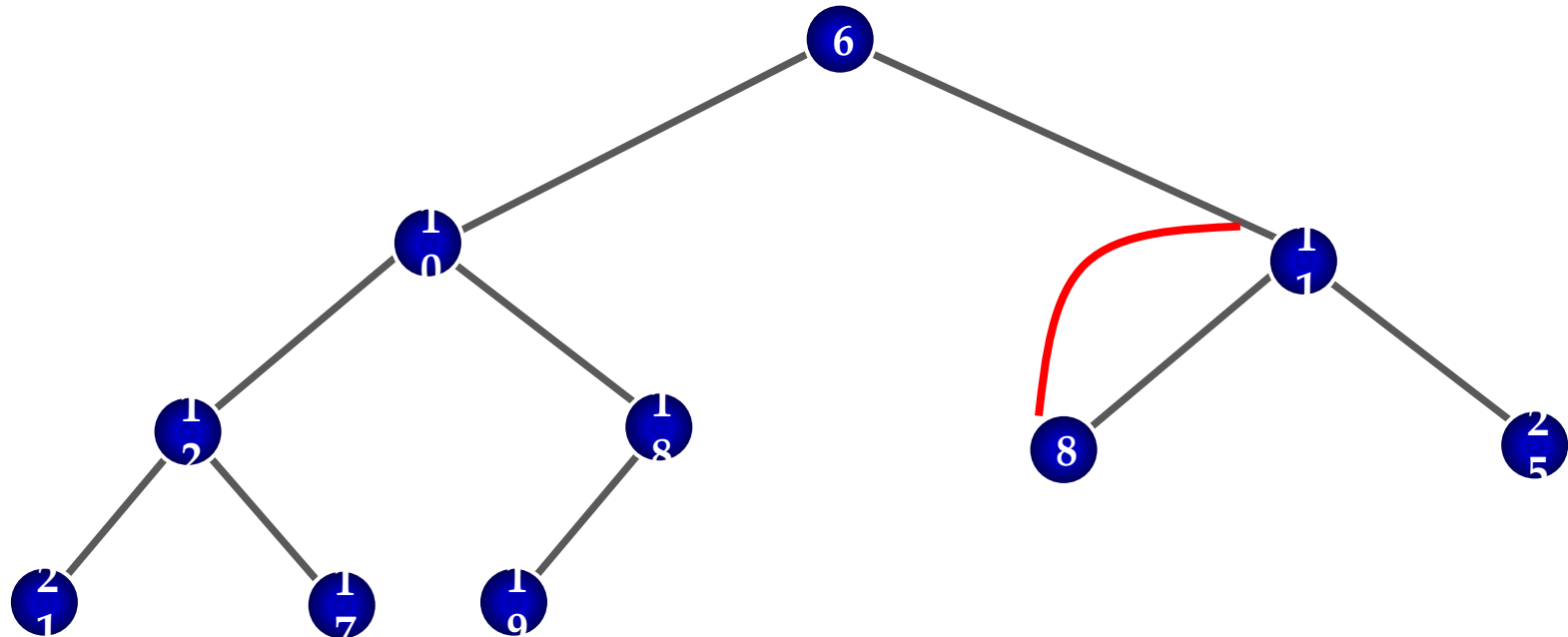
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	11	10	6	12	18	8	25	21	17	19						

Constructing a Heap using heapify



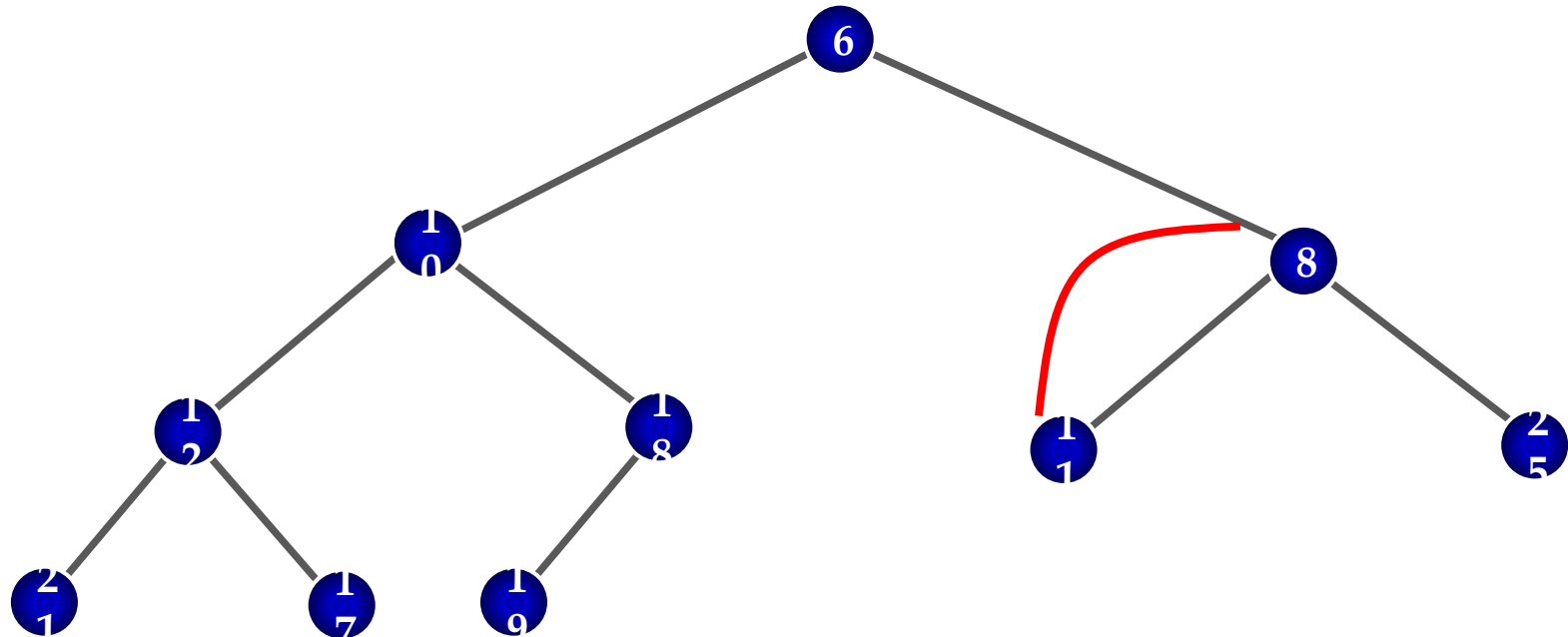
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	6	10	11	12	18	8	25	21	17	19						

Constructing a Heap using heapify



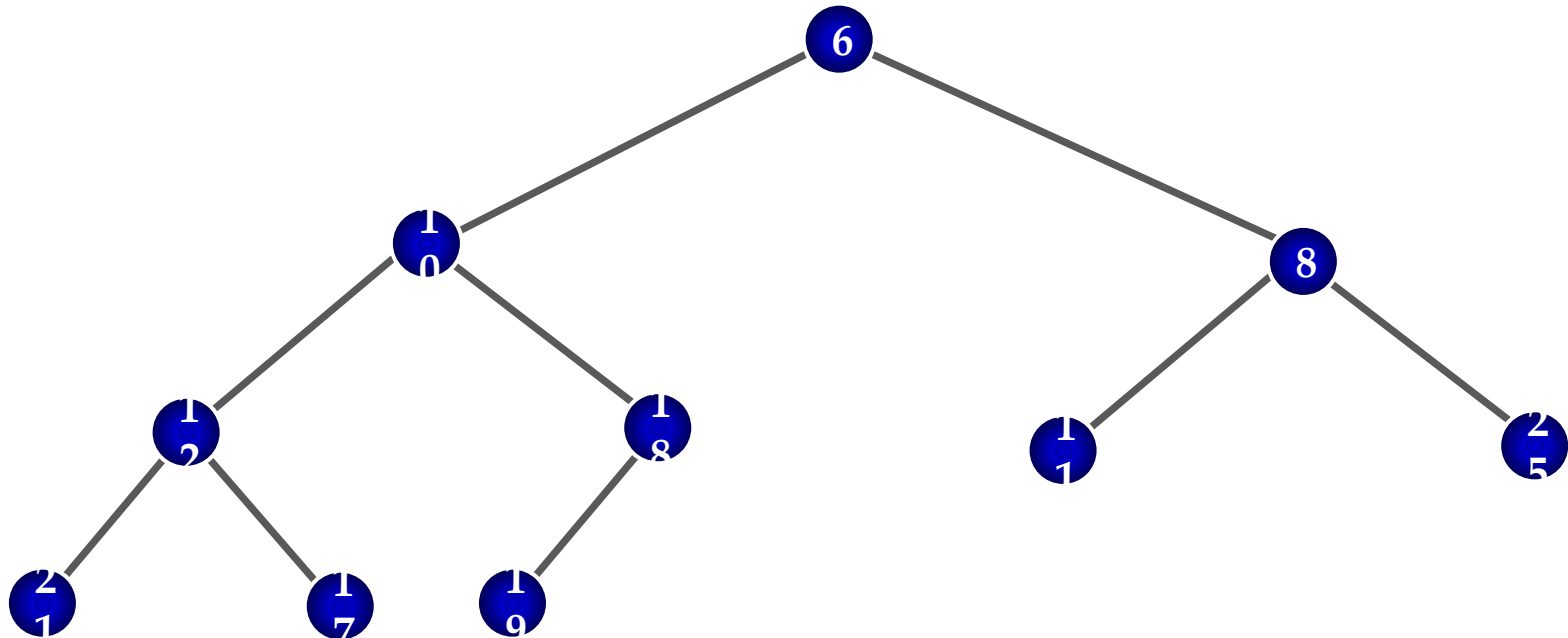
i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	6	10	11	12	18	8	25	21	17	19						

Constructing a Heap using heapify



i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	6	10	8	12	18	11	25	21	17	19						

Constructing a Heap using heapify



i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	6	10	8	12	18	11	25	21	17	19						

Build Heap: Heapify

BUILD-MIN-HEAP(A, n)

```
    for i =  $\lfloor n / 2 \rfloor$  down to 1  
        MIN-HEAPIFY(A, i, n)
```

MIN-HEAPIFY(A, i, n)

```
    left  = 2 * i  
    right = 2 * i + 1  
    smallest = i  
  
    if left  $\leq$  n and A[left] < A[smallest]  
        smallest = left  
  
    if right  $\leq$  n and A[right] < A[smallest]  
        smallest = right  
  
    if smallest  $\neq$  i  
        swap A[i] and A[smallest]  
        MIN-HEAPIFY(A, smallest, n)
```

Min Heap: Running time

Operation	Running time
Insert	$O(\log n)$
Decrease key	$O(\log n)$
Extract min	$O(\log n)$
Delete	$O(\log n)$
Find min	$O(1)$
Build heap	$O(n)$

Heapsort

- Repeatedly perform n *extract-min-like* operations. This can be done in-place
 - Running time: $O(n \log n)$

i	1	2	3	4	5	6	7	8	9	10
key	6	10	8	12	18	11	25	21	17	19



i	1	2	3	4	5	6	7	8	9	10
key	19	10	8	12	18	11	25	21	17	6



Heapify